

SEMANA 2 — DÍA 4

Integración del Script 1 (versión 0)

Objetivo del día:

Integrar conexión a la API, validación y limpieza en un solo flujo modular.

Al final de la clase, cada equipo debe tener un **Script 1 v0 funcional**, aunque no perfecto.

TEORÍA

1. ¿Qué es un flujo de datos?

Un flujo de datos es el camino que sigue la información desde que llega hasta que se transforma.

En el Script 1, el flujo es:

API → Validación → Limpieza → Guardado

✓ Explicación simple:

- **API:** obtienes datos crudos
- **Validación:** verificas que los datos tengan el formato correcto
- **Limpieza:** conviertes, normalizas, aplanas
- **Guardado:** escribes los datos limpios en un archivo

2. ¿Qué significa “versión 0”?

- No es final
- No necesita ser perfecta
- Solo debe funcionar de inicio a fin
- Puede tener validaciones mínimas
- Puede tener limpieza básica

Idea clave: *Una versión 0 es un prototipo funcional.*

3. Sintaxis y ejemplos para integrar módulos

✓ Importar funciones entre módulos

```
from api_client import get_data
from validators import validar_nombre
from cleaner import limpiar_datos
```

4. Ejemplo de flujo completo (guía, no solución)

```
from api_client import get_data
from validators import validar_nombre
from cleaner import limpiar_datos
import json

url = "https://pokeapi.co/api/v2/pokemon/ditto"

data = get_data(url)

if data:
    if validar_nombre(data["name"]):
        clean = limpiar_datos(data)
        with open("data/clean/data_clean.json", "w") as f:
            json.dump(clean, f, indent=4)
```

5. Buenas prácticas para integrar módulos

- No mezclar validación y limpieza
- No duplicar código
- Mantener funciones pequeñas
- Manejar errores en cada etapa
- Documentar cada función
- Mantener nombres consistentes

EJEMPLOS

✓ *api_client.py*

```
import requests

def get_data(url):
    try:
        response = requests.get(url)
        if response.status_code == 200:
            return response.json()
        else:
            print("Error en API:", response.status_code)
            return None
    except Exception as e:
        print("Error de conexión:", e)
        return None
```

✓ *validators.py*

```
def validar_nombre(nombre):
    return nombre.isalpha()
```

✓ *cleaner.py*

```
def limpiar_datos(data):
    clean = {
        "nombre": data["name"].capitalize(),
        "altura": int(data["height"])
    }
    return clean
```

TAREA

Objetivo: Construir el flujo completo del Script 1 FINAL, usando los módulos creados ayer.

Ayer ya lograron:

- Crear carpetas (data/, data/raw/, data/clean/, src/)
- Crear módulos base (api_client.py, validators.py, cleaner.py, utils.py)
- Agregar funciones mínimas
- Probar imports
- Crear main.py
- Probar os y sys.argv
- Subir la estructura inicial a GitHub

Hoy deben **usar esa base para avanzar directamente hacia el ENTREGABLE 2 del 26 de abril.**

¿Qué deben lograr hoy?

Construir el **primer flujo funcional del Script 1 FINAL**, que incluya:

1. Conexión a la API (módulo api_client.py)
2. Validación real con regex (módulo validators.py)
3. Limpieza y normalización (módulo cleaner.py)
4. Guardado de datos limpios (carpeta data/clean/)
5. Ejecución desde main.py
6. Subida del avance a GitHub

Instrucciones completas de la tarea

1. Conexión a la API — api_client.py

El módulo debe incluir:

- Función GET
- Manejo básico de errores
- Validación del status_code
- Retorno del JSON crudo

Ejemplo guía:

```
import requests

def get_data(url):
    try:
        response = requests.get(url)
        if response.status_code == 200:
            return response.json()
        else:
            print("Error en API:", response.status_code)
            return None
    except Exception as e:
        print("Error de conexión:", e)
        return None
```

2. Validación de datos — validators.py

Aquí deben usar **expresiones regulares** para validar **al menos 2–3 campos** de su API.

Pueden validar:

- IDs numéricos
- Fechas
- Códigos

- Textos
- Campos obligatorios

Ejemplo guía:

```
import re

def validar_nombre(nombre):
    return bool(re.search(r"^[A-Za-zÁÉÍÓÚáéíóúñÑ]+$", nombre))
Si algo no cumple:
print("Advertencia: nombre inválido")
```

3. Limpieza y normalización — *cleaner.py*

El módulo debe:

- Convertir tipos (str → int, float)
- Aplanar estructuras anidadas
- Normalizar nombres
- Crear un diccionario limpio

Ejemplo guía:

```
def limpiar_datos(data):
    clean = {
        "nombre": data["name"].capitalize(),
        "altura": int(data["height"]),
        "peso": float(data["weight"])
    }
    return clean
```

4. Guardado de datos limpios

Guardar el resultado en:

```
data/clean/data_clean.json
```

Ejemplo guía:

```
import json

def guardar_datos(clean):
    with open("data/clean/data_clean.json", "w") as f:
        json.dump(clean, f, indent=4)
```

5. Ejecutarse desde *main.py*

El archivo principal debe orquestar todo:

```
from api_client import get_data
from validators import validar_nombre
from cleaner import limpiar_datos
import json

url = "https://pokeapi.co/api/v2/pokemon/ditto"

data = get_data(url)

if data:
    if validar_nombre(data["name"]):
        clean = limpiar_datos(data)
        with open("data/clean/data_clean.json", "w") as f:
            json.dump(clean, f, indent=4)
```

6. Subir avances a GitHub

Cada equipo debe subir:

- Estructura completa del proyecto
- Módulos actualizados
- Flujo funcional del Script 1 FINAL
- Carpeta data/clean/ con el archivo generado
- Commits de todos los integrantes

Resultado esperado de la tarea

Al terminar esta actividad, cada equipo debe tener:

- ✓ Un flujo funcional del Script 1 FINAL
- ✓ Conexión → validación → limpieza → guardado
- ✓ Módulos trabajando juntos
- ✓ Datos limpios generados
- ✓ Todo subido a GitHub
- ✓ Avance directo hacia el ENTREGABLE 2



TAREA (Enfocada al ENTREGABLE 2 – 26 de abril)

Además de lo anterior, cada equipo debe:

✓ Integrar validación + limpieza en un solo flujo

Usar funciones de:

- validators.py
- cleaner.py

✓ Completar api_client.py

Debe incluir:

- función GET
- manejo básico de errores

✓ Subir avances a GitHub

- Estructura completa
- Módulos creados
- Commits de todos